

Name: ycchiu226

Date: 2026-09-24

Topic: Lists, Sorting, Aliasing, Mutation, and Algorithmic Problem Solving

1. Today's Challenge

Core concept from today's OCW lecture:

Using lists, sorting, mutation/reference management, and nested lists to solve a new problem without unintentionally modifying the original data.

AI-generated coding challenge title:

Grouped Color Report

2. My Initial Approach — Before AI Help

Before asking AI for hints, briefly describe how you planned to solve the problem.

My approach:

First, I planned to traverse through the nested list and flatten all the color strings into one list. Then I would sort the flattened list alphabetically.

After that, I planned to traverse the sorted list one color at a time. For each color, I would check the next entry to see whether it was identical. If it was, I would continue counting. If it was different, I would know that I had reached the end of that color's group, so I could create a tuple containing the color and its appearance count and append it to the result.

3. AI Tutor Help

Did you ask the AI Tutor for help?

☐ No — I solved it independently

☒ Yes — I received one or more hints

The most useful hint/question from AI was:

The AI asked me how I would make sure that after counting a color, I would not count that same color again when the outer loop reached its next occurrence. It suggested thinking about either maintaining a

separate list of already processed colors or using the fact that identical colors become adjacent after sorting.

It helped me realize that:

I needed to carefully manage the index while counting consecutive identical colors. The inner loop should advance the index through all occurrences of the current color, so the outer loop resumes at the first unprocessed color instead of starting again from the second or third occurrence of the same color.

4. My Revision

Did I change my approach or code after interacting with AI?

- ☐ No
- ☒ Yes

What did I change, and why?

I refined my counting algorithm. I used a variable to record the color I was currently counting and used an inner loop to compare the next entry with that color. Whenever the next entry was identical, I increased the count and advanced the index. When the next entry was different, I created the `(color, count)` tuple and appended it to the result.

I also made sure that the index advanced past all occurrences of the current color, so the same color would not be processed again.

5. Verification

My final program:

- ☒ Passed the provided examples
- ☒ Passed additional edge cases
- ☐ Still has unresolved problems

One edge case I tested:

Input:

`[]` (empty input)

Expected output:

[]

Actual output:

[]

I also considered cases with only one color, multiple identical colors, and aliased inner lists.

6. One-Minute Reflection

What idea from the OCW lecture did I transfer to this new problem?

I transferred the idea of using sorting and careful list/reference management to simplify the counting problem while avoiding unintended mutation of the input. Sorting put identical colors next to each other, which allowed me to count them with a linear traversal. I also created a separate flattened list so that sorting it would not modify the original nested list.

One thing I understand better now:

I understand better how sorting can be used as part of an algorithm to change the structure of a problem and make the remaining work simpler. Instead of searching through the entire list for every color, I can sort the data first and then count consecutive identical elements in one traversal. I also understand more clearly how careful index management prevents already-processed elements from being processed again.

One thing I am still unsure about:

I am still somewhat unsure about how the time complexity of different Python list operations should be determined in more complicated programs, especially when several loops and list operations are combined. I understand that the sorting step here dominates and gives an overall complexity of $O(N \log N)$, but I want more practice analyzing similar cases.